

Chapter 7

Vision Transformers

Exam Study Guide, Worked Concepts, Sample Questions, and

Prepared from	uploaded course materials
Main source	Chapter_7_Vision_Transformers.ipynb
Exam source	Chapter_7_Summary_and_Sample_Questions.pdf
Practice source	Chapter_7._Flashcardspdf.pdf

How to use it

Study ViT as the Transformer idea applied to images. The key move is turning an image into a sequence of patch tokens, adding positions, and using a Transformer encoder.

Study order: First read the concept sections, then solve the five sample exam questions, then test yourself with the flashcards.

Contents

- 1. Quick exam map
- 2. Why Vision Transformers matter
- 3. From image to token sequence
- 4. Patch embedding
- 5. Positional information
- 6. Class token and pooling
- 7. ViT blocks and CNN comparison
- 8. Formula cheat sheet and symbol key
- 9. Code prompts
- 10. Exam-style questions
- 11. Flashcards

Most useful mental shortcut

A ViT is: image $\rightarrow$ patches $\rightarrow$ patch embeddings + positions + optional class token $\rightarrow$ Transformer encoder $\rightarrow$ task head.

1. Quick exam map

Vision Transformers process images as sequences of visual tokens, making image modeling compatible with Transformer-style attention.

Image $H \times W$ raw pixels Patches $P \times P$ non-overlapping visual chunks

Patch

embeddings

tokens in d

dimensions Position + class token spatial and global information

Transformer

encoder contextualized image representation Figure: the main method pipeline for this chapter.

Exam topic	What you must be able to say
Patch tokenization	Split image into non-overlapping patches and map each to an embedding.
Patch count	Number of tokens grows as $(H/P)(W/P)$.
Position embeddings	Needed because attention alone lacks spatial order.
Class token	Dedicated sequence position for global image-level information.
ViT block	Transformer encoder block with self-attention and MLP.

CNN vs. ViT CNN has local inductive bias; ViT learns flexible global interactions.

2. Zero-prior explanation: understand the

idea before the equations This section assumes you have not read the lecture yet. The goal is to build intuition first, then connect the intuition to the equations and exam language.

Why ViTs matter

ViTs showed that strong image representation can be learned by tokenizing images and applying Transformer encoders. This connects vision to the same sequence machinery used in language models.

From image to patch sequence

An image is divided into patches. Each patch is flattened and projected to a vector, so the image becomes a sequence of tokens.

Patch embedding

Patch embedding can be a linear projection of flattened patches or a convolution with kernel size and stride equal to the patch size.

Memory line
ViT makes an image look like a sentence of patch tokens.

3. Core concepts and worked explanations

Positional information

Without position embeddings, attention knows patch content but not where patches came from.

Images are spatial, so position information is essential.

Class token and heads

A class token is a learnable token that gathers global information through attention.

Classification uses its final representation, but mean pooling can also be used.

Strengths and limitations

ViTs model global relationships flexibly and fit multimodal systems well. However, attention cost grows quadratically with token count, and ViTs often need more data or pretraining than CNNs.

4. Method explanation, worked example,

and application

Step	Method explanation
Patchify image	Split $H \times W$ image into $P \times P$ patches.
Project patches	Flatten and map each patch into a vector token.
Add spatial context	Add positional embeddings and optional class token.
Encode with attention	Let patches exchange information globally.

Worked example

Example: a 224×224 image with 16×16 patches gives $14 \times 14 = 196$ patch tokens. A class token can attend to all patches and become the final image-level representation.

Applications and usage

Usage: ViTs are used in image classification, CLIP image encoders, masked image modeling, diffusion transformers, and multimodal systems.

5. Formula / definition cheat sheet

Read formulas as compact sentences. Do not memorize symbols blindly; use the symbol key table after the formula table to understand what each part represents.

Formula	Interpretation
$N = (H/P)(W/P)$ Number of image patches.	x_p^i E Patch embedding.
$z_0 = [x_{\text{class}}; x_p^1 E; \dots; x_p^N E] + \text{Epos Initial ViT token sequence.}$	$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d})V$ Self-attention over patches.

$\hat{y} = \text{MLP}(z_L_{\text{class}})$ Image-level classification route.

6. Equation symbol key table

This table explains the main symbols used in the equations. When studying, cover the meaning column and check whether you can explain each symbol clearly.

Symbol	Meaning	Plain-English reading
H, W	Image height and width	Spatial dimensions of image.
P	Patch size	Side length of each square patch.
N	Number of patches	How many visual tokens are produced.
E	Patch projection	Linear map from patch pixels to token
vector.	Epos	Position embedding
Vector telling the model where a patch	came from.	x_{class}
Class token	Learned global summary token.	z_L_{class}
Final class-token state	Image-level representation after L	layers.

7. Main code prompt / implementation

skeleton This is not meant to replace the lecture code. It is a compact prompt showing the computational pattern you should recognize in homework and exams.

```
# ViT patch embedding prompt
```

```
patches = split_image_into_patches(image, patch_size=P)
```

```
tokens = linear_projection(flatten(patches))
```

```
tokens = add_class_token(tokens)
```

```
tokens = tokens + position_embedding
```

```
features = transformer_encoder(tokens)
```

```
logits = classifier(features[:, 0])
```

How to explain the code in words

Start by identifying the input, the model output, the objective being minimized, and the update rule. Then connect the code back to the formulas and the method table.

8. Common exam traps

Trap	Correct way to think
Thinking ViT sees raw pixels	directly It sees patch embeddings.
Ignoring patch size	Patch size controls token count and cost.
Saying ViTs always beat CNNs	Performance depends on data, compute, and task.

9. Exam-style sample questions with

answers These are written in the style of conceptual exam answers. Use them to practice explaining, not only recognizing terms.

Question 1

Can a ViT classify without a class token?

Sample answer

Yes. It can pool patch tokens, but the class token provides a dedicated global summary path.

Question 2

Why are positions needed?

Sample answer

Self-attention alone is permutation-invariant and does not know spatial layout.

Question 3

Why is patch size a tradeoff?

Sample answer

Smaller patches preserve detail but increase sequence length and attention cost.

Question 4

Are ViTs always better than CNNs?

Sample answer

No. CNNs have useful local biases and can be more data-efficient.

Question 5

Why are ViTs important for VLMs?

Sample answer

They convert images into token embeddings that can align or fuse with language tokens.

10. Flashcards for rapid review

Use these for active recall. Cover the answer column and try to answer before checking.

Card 1

Q: What is a patch token?

A: Embedding of an image patch.

Card 2

Q: How many 16×16 patches in 224×224 ?

A: 196.

Card 3

Q: Why add position embeddings?

A: To encode spatial layout.

Card 4

Q: What is the class token?

A: A global summary token.

Card 5

Q: What does attention allow?

A: Global patch interactions.

Card 6

Q: What is ViT limitation?

A: Quadratic attention cost.

Card 7

Q: What does patch embedding do?

A: Converts pixels to tokens.

Card 8

Q: What is mean pooling alternative?

A: Average final patch tokens.

Card 9

Q: What must you remember about Patch tokenization?

A: Split image into non-overlapping patches and map each to an embedding.

Card 10

Q: What must you remember about Patch count?

A: Number of tokens grows as $(H/P)(W/P)$.

Card 11

Q: What must you remember about Position embeddings?

A: Needed because attention alone lacks spatial order.

Card 12

Q: What must you remember about Class token?

A: Dedicated sequence position for global image-level information.

Card 13

Q: What must you remember about ViT block?

A: Transformer encoder block with self-attention and MLP.

Card 14

Q: What must you remember about CNN vs. ViT?

A: CNN has local inductive bias; ViT learns flexible global interactions.

Card 15

Q: What does this formula represent:

$N=(H/P)(W/P)$?

A: Number of image patches.

Card 16

Q: What does this formula represent: x_p^{iE} ?

A: Patch embedding.

Card 17

Q: What does this formula represent: $z_0=[x_{class}; x_p^{1E}; \dots; x_p^{NE}] + E_{pos}$?

A: Initial ViT token sequence.

Card 18

Q: What does this formula represent:

$\text{Attention}(Q,K,V)=\text{softmax}(QK^T/\sqrt{d})V$?

A: Self-attention over patches.

Card 19

Q: What does this formula represent:

$\hat{y}=\text{MLP}(z_{L_class})$?

A: Image-level classification route.

Card 20

Q: What does H,W mean?

A: Image height and width. Spatial dimensions of image.

Card 21

Q: What does P mean?

A: Patch size. Side length of each square patch.

Card 22

Q: What does N mean?

A: Number of patches. How many visual tokens are produced.

Card 23

Q: What does E mean?

A: Patch projection. Linear map from patch pixels to token vector.

Card 24

Q: What does Epos mean?

A: Position embedding. Vector telling the model where a patch came from.

Card 25

Q: What does xclass mean?

A: Class token. Learned global summary token.

Card 26

Q: What does zL_class mean?

A: Final class-token state. Image-level representation after L layers.

Card 27

Q: What is the key idea of Why ViTs matter?

A: ViTs showed that strong image representation can be learned by tokenizing images and applying Transformer encoders. This connects vision to the same sequence machinery used in language

Card 28

Q: What is the key idea of From image to patch sequence?

A: An image is divided into patches. Each patch is flattened and projected to a vector, so the image becomes a sequence of

Card 29

Q: What is the key idea of Patch embedding?

A: Patch embedding can be a linear projection of flattened patches or a convolution with kernel size and stride equal to the patch

Card 30

Q: What is the key idea of Positional information?

A: Without position embeddings, attention knows patch content but not where patches came from. Images are spatial, so position information is

Card 31

Q: What is the key idea of Class token and heads?

A: A class token is a learnable token that gathers global information through attention. Classification uses its final representation, but mean pooling can also be

Card 32

Q: What is the key idea of Strengths and limitations?

A: ViTs model global relationships flexibly and fit multimodal systems well. However, attention cost grows quadratically with token count, and ViTs often need more data or pretraining than

Card 33

Q: What happens in the step: Patchify image?

A: Split $H \times W$ image into $P \times P$ patches.

Card 34

Q: What happens in the step: Project patches?

A: Flatten and map each patch into a vector token.

Card 35

Q: What happens in the step: Add spatial context?

A: Add positional embeddings and optional class token.

Card 36

Q: What happens in the step: Encode with attention?

A: Let patches exchange information globally.

Card 37

Q: Can you describe the ViT pipeline?

A: Image, patches, embeddings, positions, encoder, head.

Card 38

Q: Can you calculate patch count?

A: $(H/P)(W/P)$.

Card 39

Q: Can you compare CNN and ViT?

A: Local bias vs flexible attention.

Card 40

Q: Can you explain class token?

A: Dedicated global representation.

Card 41

Q: Can a ViT classify without a class token?

A: Yes. It can pool patch tokens, but the class token provides a dedicated global summary path.

Card 42

Q: Why are positions needed?

A: Self-attention alone is permutation-invariant and does not know spatial layout.

Card 43

Q: Why is patch size a tradeoff?

A: Smaller patches preserve detail but increase sequence length and attention cost.

Card 44

Q: Are ViTs always better than CNNs?

A: No. CNNs have useful local biases and can be more data-efficient.

Card 45

Q: Why are ViTs important for VLMs?

A: They convert images into token embeddings that can align or fuse with language tokens.

Card 46

Q: In plain English, why is $N=(H/P)(W/P)$ important?

A: It captures: Number of image patches. Use it as a compact exam sentence, then define each symbol.

Card 47

Q: In plain English, why is $x_p^i \in E$ important?

A: It captures: Patch embedding. Use it as a compact exam sentence, then define each symbol.

Card 48

Q: In plain English, why is $z_0 = [x_{\text{class}}; x^{1E}; \dots; x^{NE}] + \text{Epos}$ important?

A: It captures: Initial ViT token sequence. Use it as a compact exam sentence, then define each symbol.

Card 49

Q: In plain English, why is $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d})V$ important?

A: It captures: Self-attention over patches. Use it as a compact exam sentence, then define each symbol.

Card 50

Q: In plain English, why is $\hat{y} = \text{MLP}(z_{L_{\text{class}}})$ important?

A: It captures: Image-level classification route.

Use it as a compact exam sentence, then define each symbol.

Card 51

Q: In plain English, why is $N = (H/P)(W/P)$ important?

A: It captures: Number of image patches. Use it as a compact exam sentence, then define each symbol.

Card 52

Q: In plain English, why is x^{iE} important?

A: It captures: Patch embedding. Use it as a compact exam sentence, then define each symbol.

Card 53

Q: In plain English, why is $z_0 = [x_{\text{class}}; x^{1E}; \dots; x^{NE}] + \text{Epos}$ important?

A: It captures: Initial ViT token sequence. Use it as a compact exam sentence, then define each symbol.

Card 54

Q: In plain English, why is $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d})V$ important?

A: It captures: Self-attention over patches. Use it as a compact exam sentence, then define each symbol.

Card 55

Q: In plain English, why is $\hat{y} = \text{MLP}(z_{L_{\text{class}}})$ important?

A: It captures: Image-level classification route.

Use it as a compact exam sentence, then define each symbol.

Explanation

Chapter 7: Vision Transformers

How this section is organized

Each item below is a heading taken from the corresponding Jupyter notebook in the same order it appears there. The explanation beside it tells you what that notebook section is trying to teach and why it matters for the review.

Source notebook

Chapter_7_Vision_Transformers.ipynb

Headings included

22

Notebook headings with explanation

1. Chapter 7 Vision Transformers	After this chapter you should be able to: 1. Explain the motivation for Vision Transformers (ViTs).
2. Why Vision Transformers Matter for Modern Generative Models	Vision Transformers (ViTs) began as a discriminative architecture for image recognition: the central idea is to split an image into fixed-size patches, linearly embed those patches, add positional information, and then process the resulting sequence with a standard Transformer encoder. In other words, a ViT reframes an image as a sequence of visual tokens.
3. A short history of Vision Transformers	The modern Vision Transformer did not appear in isolation. It emerged from a sequence of ideas that gradually moved the transformer from language sequences to vision representations.
4. What is a Vision Transformer?	https://viso.ai/deep-learning/vision-transformer-vit/ This diagram shows how a Vision Transformer (ViT) converts an image into a prediction. The key ideas in the architecture are described below.
5. Task dimension: classification vs. encoding, and why the class token is added	A Vision Transformer can be used for more than one kind of task. The most important distinction is between: 1.
6. Positional encodings in Vision Transformers	Transformers are permutation-invariant, so a Vision Transformer (ViT) needs positional information to know where each patch came from. In the original ViT, this was handled with learned absolute positional embeddings added to the patch tokens before the transformer blocks.
7. Building a Vision Transformer for CIFAR10	We will build a vision transformer for CIFAR10 to demonstrate the various architectural elements of a Vision Transformer. The notebook also includes 3 nearby code cells for implementation, demonstration, or visualization of this heading.
8. CIFAR-19 Data	We download CIFAR-10 and create standard training and test loaders. For training, we use light augmentation: - random crop with padding - random horizontal flip - normalization For evaluation, we only normalize. The notebook also includes 2 nearby code cells for implementation, demonstration, or visualization of this heading.
9. Visualize a few training images	The tensors are normalized, so for display we undo the normalization. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.

10. Patch embedding	The patch embedding uses a clever trick of applying a convolutional layer without an activation function to the original image in a manner so that the output of application of each filter represents one dimension of the token projection. So each input patch is run through all convolutional filters as the linear projections. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
11. Attention, MLP, and Transformer block	Each transformer block contains: 1. Residual connection For attention, the standard formula is $\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^{\text{top}}}{\sqrt{d_h}})V$ where d_h is the dimension per attention head. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
12. ViT encoder with class token and positional embeddings	The encoder does the following: 1. convert the image into patch tokens 2. prepend a learned class token 3. add learned positional embeddings 4. process the sequence with transformer blocks 5. normalize the final sequence If the patch tokens are X_{patch} in $R^{B \times N \times D}$ and the class token is x_{cls} in $R^{1 \times 1 \times D}$ then the full input sequence becomes X_0 in $R^{B \times (N+1) \times D}$ The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
13. Classification head	We use the output corresponding to the class token which is at position 0. So z contains the CLS tokens for the whole batch. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
14. Instantiate the model	These settings are intentionally moderate for CIFAR-10: - patch_size = 4 - embed_dim = 192 - depth = 6 - num_heads = 3 That is large enough to be a real ViT example, but still manageable. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
15. Quick shape check	For CIFAR-10, with patch size 4: $N = (32/4)^2 = 64$ After adding the class token, the transformer sees a sequence of length: $N + 1 = 65$ The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
16. Optimizer, loss, and learning-rate schedule	We use: - cross-entropy loss - AdamW optimizer - cosine annealing scheduler The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
17. Training and evaluation loops	This notebook heading introduces training and evaluation loops and explains how it supports the chapter concepts, equations, method steps, or implementation workflow. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
18. Train the model	A small number of epochs is enough to demonstrate the pipeline. If you are on GPU, you can increase the epoch count. The notebook also includes 2 nearby code cells for implementation, demonstration, or visualization of this heading.
19. Plot training curves	This notebook heading introduces plot training curves and explains how it supports the chapter concepts, equations, method steps, or implementation workflow. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.
20. Show a few predictions	This notebook heading introduces show a few predictions and explains how it supports the chapter concepts, equations, method steps, or implementation workflow. The notebook also includes 1 nearby code cell for implementation, demonstration, or visualization of this heading.

21. Summary

This example shows that the same ViT-style encoder can be used for image classification by adding a simple classification head. The flow is: $[B, 3, 32, 32] \rightarrow [B, 64, D] \rightarrow [B, 65, D] \rightarrow [B, D] \rightarrow [B, 10]$ where: - 64 comes from the 8 times 8 patch grid for $\text{patch_size} = 4$ - 65 includes the extra class token - the final output is one logit vector per image This is also a useful bridge to Vision-Language Models: - for classification, the image encoder feeds a classifier - for CLIP/SigLIP, the image encoder.

22. Comparing CNN vs ViT for classification and generation

It is interesting to compare CNN vs. ViT for classification and generation tasks.